

Function $\langle T, R \rangle$

= primește o informație și o transformă în altceva

= este baza pt. mapare / transformare

* Semnatura: $R \text{ apply } (T*)$

tipul de
ieșire

tipul de intrare

exemplu ①: Function care returnează lungimea unui String

Function $\langle \text{String}, \text{Integer} \rangle$ lungime = $s \rightarrow s.length()$;

len = lungime.apply("capac");

$\Rightarrow \text{len} = 5$

ex ②: la fel, cu referință la metodă

Function $\langle \text{String}, \text{Integer} \rangle$ lungime = $\text{String}::\text{length}$;

len = lungime.apply("carte");

ex ③: cu andThen $\rightarrow$ leagă două funcții, adică

rezultatul primei funcții devine intrarea celei de-a doua funcții

* facem o funcție care ia un String și îl convertește în Integer, apoi ia acel număr și îl înmulțește cu 10.

prima funcție: Function $\langle \text{String}, \text{Integer} \rangle$ convertește
= $\text{Integer}::\text{parseInt}$;

a doua funcție: Function $\langle \text{String}, \text{Integer} \rangle$

a doua functie:

Function <Integer, Integer> immultire = (x) → x * 10;

a treia functie:

Function <~~Int~~ String, Integer> finala =
= convertire. andThen(imultire);

Supplier <T>

→ nu primește nimic, returnează ceva

metoda abstractă: **T get()**

Supplier <Double> generator = () → Math.random();

↙
nu primește nimic,
dar returnează un nr.
random.

Double r = generator.get();

Supplier <String> furnizor = () → "Invat Java 8";

Comparator

→ scop: compara 2 obiecte pentru a le putea sorta.

* Metodă abstractă: `int compare(T o1, T o2)`

↳ returnează 0 dacă $o1 = o2$
negativ dacă $o1 < o2$
pozitiv dacă $o1 > o2$

exemplu (model vechi, fără lambda):

```
Comparator<Student> comp = new Comparator<Student>() {  
    public int compare(Student s1, Student s2) {  
        return s1.getNota().compareTo(s2.getNota());  
    }  
}
```

* cu funcție lambda:

```
Comparator<Student> comp = (s1, s2) -> s1.getNota().  
    .compareTo(s2.getNota());
```

Exerciții din curs (cu comparator)

```
public class StudentHelper {
```

```
    public static int compareByAverage(Student a, Student b) {  
        return (int) (a.getMedia() - b.getMedia());  
    }
```

```
    public static int compareByName(Studenta s1, Studentb s2) {  
        return (int) a.getNume().compareTo(b.getNume());  
    }
```

```
}
```

① folosiți clasa StudentHelper și definiți comparatori pentru sortarea unei liste de (studenți).

Folosiți ~~la~~ referință la metodă.

```
Comparator<Student> comparator_dupa_medie  
= StudentHelper::compareByAverage;
```

și apoi sortăm cu lista.sort (comparator_dupa_medie);

② sortați o listă de studenți, definind comparatori ca funcții lambda.

```
lista.sort((s1, s2) -> s1.getName().compareTo(s2.getName()));
```

↳ va sorta lista direct

Optional - nu este interogată funcțională!!!

→ este un container care ține o valoare sau nu ține nimic

→ Prevină NullPointerException

Sintaxă: Optional.empty() → creează o cutie goală.

Optional.of(valoare) → pune valoarea în cutie
Dacă valoarea dată este null ⇒
⇒ va arunca NullPointerException.

Optional.ofNullable(valoare) → cea mai sigură

Dacă valoarea e null ⇒ Creează un Optional gol.

Dacă valoarea există ⇒ creează un optional cu valoarea în el.

Extragerea din Optional

~~ifPresent()~~ verifică
dacă e acolo ceva
(caușușușu)

`opt. get()` → not safe

`opt. orElse(altceva)` → scoate valoarea
dacă nu e nimic ⇒ returnează
valoarea altceva (valoare default)

`opt. orElseThrow(...)` → scoate valoarea
dacă nu este ⇒ throw eroarea
asta

* exemplu: avem un String s, care s-ar putea să fie Null.

⇒ `Optional <String> sir = bun (String s)`

`return Optional.ofNullable(s);`

}

↓
dacă s e text ⇒ îl pune în cutie
dacă s e null ⇒ face o cutie goală.

și pentru desfacere →

① `if (sir.bun.isPresent())`
↓
`System.out.println(sir.bun.get());`

~~② `sir.bun.isPresent()`~~

② `sir.bun.ifPresent(System.out::println);`

③ `sir.bun.ifPresent(x) → System.out::println(x);`